

DATA ANALYSIS USING ADVANCED SQL

PRESENTATION BY: MANTSHA SHEIKH

JENSON USA



Welcome To Jenson USA Presentation

Jenson USA is a leading online bicycle retailer in the United States, founded in 1994. The company offers a wide range of bicycles, cycling components, accessories, and apparel for riders across multiple categories. This dataset helps analyze sales performance, customer behavior, and product demand using SQL.

Dataset Overview

Dataset Overview – Jenson USA

- Brands – Contains information about different bicycle brands available in the store.
- Categories – Stores product category details such as types of bikes and accessories.
- Customers – Includes customer personal and contact information for purchase analysis.
- Orders – Records order transaction details like order dates and customer purchases.
- Order_Items – Provides product-level details within each order including quantity and price.
- Products – Contains product names, categories, and pricing information.
- Staffs – Includes employee details responsible for handling customer orders.
- Stocks – Shows inventory availability of products across different stores.
- Stores – Stores information about store locations and their operations.



Purpose of the Dataset

- Analyze sales performance across stores
- Understand customer purchasing behavior
- Identify top-selling products and categories
- Evaluate staff performance
- Monitor inventory levels across stores

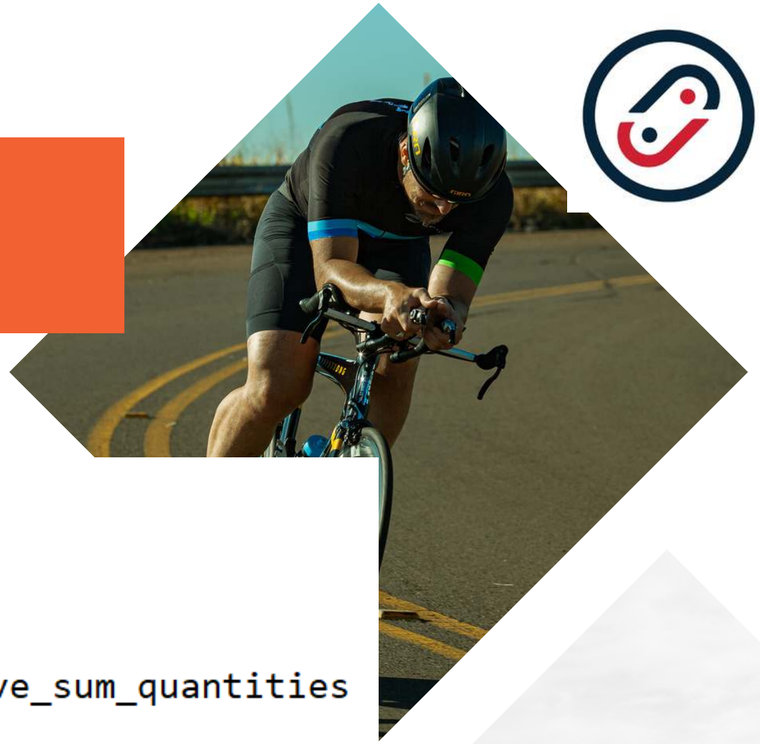




1.Find the total number of products sold by each store along with the store name.

```
2•  SELECT
3       stores.store_name,
4       SUM(order_items.quantity) total_quantity_sold
5   FROM
6       stores
7           JOIN
8       orders USING (store_id)
9           JOIN
10      order_items USING (order_id)
11  GROUP BY 1;
```

2.Calculate the cumulative sum of quantities sold for each product over time.



```sql
select order_items.product_id,
products.product_name,
orders.order_date,
order_items.quantity,
sum(quantity)over(partition by product_id order by order_date) as cumulative_sum_quantities
from orders join order_items
on orders.order_id=order_items.order_id
join products on order_items.product_id=products.product_id;
```



3.Find the product with the highest total sales (quantity * price) for each category.



```
24 • with a as( select products.product_id,
25   products.product_name,
26   products.category_id,
27   sum(order_items.quantity*order_items.list_price)sales
28   from order_items join products
29   using(product_id) group by 1,2,3)
30
31   select *from
32   (select *,rank()over(partition by category_id order by sales desc) as rnk from a)b
33   where rnk=1;
```

4.Find the customer who spent the most money on orders

```sql
SELECT
    customers.customer_id,
    CONCAT(customers.first_name,
            ' ',
            customers.last_name) AS customer_name,
    SUM(order_items.quantity * order_items.list_price) AS total_spent
FROM
    customers
        JOIN
    orders USING (customer_id)
        JOIN
    order_items USING (order_id)
GROUP BY customers.customer_id , customer_name
ORDER BY total_spent DESC
LIMIT 1;
```

5.Find the highest-priced product for each category name.

```sql
select *from (select categories.category_id,
categories.category_name,
products.product_name,
products.list_price,
dense_rank() over(partition by categories.category_id order by products.list_price desc) as rank_of_products
from products join categories
on products.category_id=categories.category_id) as a
where rank_of_products=1;
```



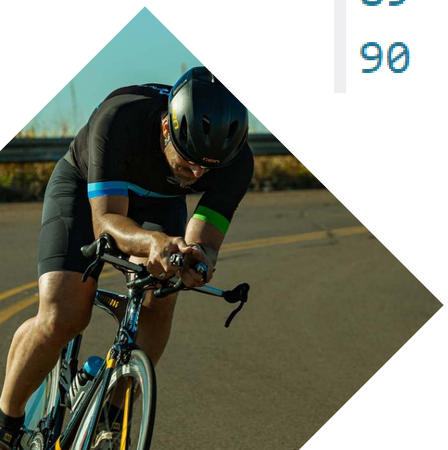


6.Find the total number of orders placed by each customer per store

```
63 • SELECT
64       orders.customer_id,
65       CONCAT(customers.first_name,
66               ' ',
67               customers.last_name) full_name,
68       store_id,
69       stores.store_name,
70       COUNT(order_id) total_orders
71   FROM
72       orders
73           JOIN
74       customers USING (customer_id)
75           JOIN
76       stores USING (store_id)
77   GROUP BY 1 , 2 , 3 , 4
78   ORDER BY customer_id;
```

7.Find the names of staff members who have not made any sales.

```
81 •  SELECT
82        staffs.staff_id,
83        CONCAT(staffs.first_name, ' ', staffs.last_name) AS staff_name,
84        orders.order_id
85    FROM
86        staffs
87            LEFT JOIN
88        orders USING (Staff_id)
89    WHERE
90        orders.staff_id IS NULL;
```

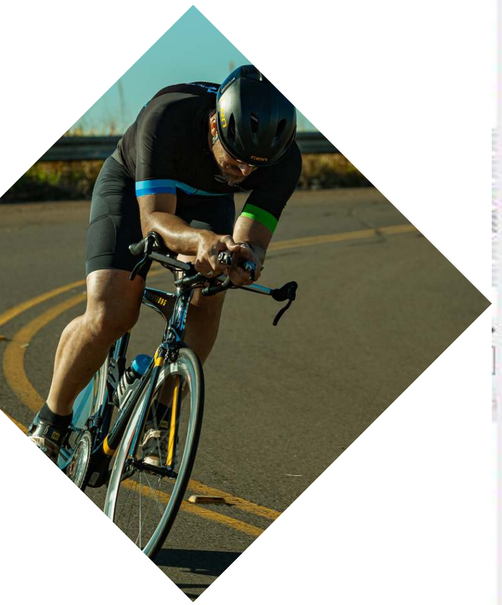


8.Find the top 3 most sold products in terms of quantity.



```
93 •   with a as
94           (select products.product_id,products.product_name,
95           sum(order_items.quantity)as quantity_sold
96           from products join order_items
97           using(product_id)
98           group by products.product_id,products.product_name),
99     b as (Select *,
00           dense_rank()over(order by quantity_sold desc)
101          as top_rank from a)
102    select* from b
103          where top_rank <=3;
```

9.Find the median value of the price list.



```
106 •  with a as
107          (select list_price,
108          row_number() over(order by list_price) as row_num,
109          count(list_price)over() as n
110          from order_items)
111    select case
112           when n% 2=0
113           then
114           (select avg(list_price) from a where row_num in((n/2),(n/2)+1))
115           else
116           (select list_price from a where row_num=(n+1)/2)
117           end as
118    median from a
119    limit 1;
```

10.List all products that have never been ordered.(use Exists)



```
123 •  SELECT
124        products.product_id, products.product_name
125    FROM
126        products
127    WHERE
128        NOT EXISTS( SELECT
129                1
130            FROM
131                order_items
132            WHERE
133                order_items.product_id = products.product_id);
```

11.List the names of staff members who have made more sales than the average number of sales by all staff members.

```sql
with a as
        (select staffs.staff_id,
        concat(staffs.first_name," ", staffs.last_name) as staff_name,
        coalesce(sum(order_items.quantity*order_items.list_price),0) as sales from staffs
        left join orders using(staff_id)
        left join order_items using(order_id)
        group by 1,2)

 select *from a
 where sales>(select avg(sales) from a)
;
```



12.Identify the customers who have ordered all types of products (i.e., from every category).



```
149 •  SELECT
150        customers.customer_id,
151        CONCAT(customers.first_name,
152                ' ',
153                customers.last_name) AS customers_name
154    FROM
155        customers
156            JOIN
157        orders USING (customer_id)
158            JOIN
159        order_items USING (order_id)
160            JOIN
161        products USING (product_id)
162            JOIN
163        categories USING (category_id)
164    GROUP BY customers.customer_id , customers_name
165    HAVING COUNT(DISTINCT categories.category_id) = (SELECT
166            COUNT(category_id)
167        FROM
168            categories);
```

Key Insights – Jenson USA SQL Project

1. Store Performance -- Different stores contribute differently to total product sales.

2. Product Demand Trend -- Cumulative product sales over time show how demand increases across different periods.

3. Category-wise Best Products -- Each category has a top-performing product generating the highest revenue.

4. Premium Product -- Each category contains high-priced products representing premium offerings.

5. Top-Selling Productst -- Top 3 products contribute significantly to total quantity sold.

JENSON USA



Thank You!
thats all from my side.